Análise de Desempenho em OpenMP

Sincronização, Falso Compartilhamento e Funções rand/rand_r na Estimativa de π

Werbert Arles de Souza Barradas

Universidade Federal do Rio Grande do Norte (UFRN)
Disciplina de Programação Paralela - DCA3703

11 de setembro de 2025

Introdução

- O projeto explora a estimativa de π usando o método de Monte Carlo, uma tarefa inerentemente paralelizável.
- O objetivo é analisar o impacto no desempenho de diferentes estratégias de sincronização e o uso de funções seguras para threads (*thread-safe*).
- O estudo busca responder a questões fundamentais sobre o paralelismo:
  - Qual o custo de usar uma função não thread-safe como `rand()`?
  - Qual o trade-off entre um bloqueio de software (`critical`) e uma abordagem com vetor compartilhado?
  - Como um problema de hardware, o **falso compartilhamento**, afeta a performance?

Metodologia: O Algoritmo Base

Método de Monte Carlo para Estimação de π

A lógica consiste em gerar milhões de pontos aleatórios (x, y) dentro de um quadrado e contar quantos caem dentro de um círculo inscrito.

- A área do círculo é πr^2 e a do quadrado é $(2r)^2 = 4r^2$.
- A razão das áreas é $\frac{\pi r^2}{4r^2} = \frac{\pi}{4}$.
- Portanto, $\pi \approx 4 \times \frac{\text{pontos no círculo}}{\text{total de pontos}}$.

Desafio

Como somar os "pontos no círculo" de forma eficiente e correta quando múltiplas threads estão trabalhando ao mesmo tempo?

Metodologia: As Quatro Versões Analisadas

Foram implementadas quatro versões para isolar diferentes gargalos:

Usando `rand()` (Não Thread-Safe)

- **Versão 1:** Acumula o total com `#pragma omp critical`.
- **Versão 2:** Cada thread escreve em um vetor compartilhado.

Usando `rand_r()` (Thread-Safe)

- **Versão 3:** Acumula o total com `#pragma omp critical`.
- **Versão 4:** Cada thread escreve em um vetor compartilhado.

Análise dos Mecanismos Utilizados

- `#pragma omp parallel for`: Usado para distribuir as 100 milhões de iterações do laço entre as threads disponíveis.
- `#pragma omp critical`: Garante que apenas uma thread por vez atualize o contador global. É uma solução de software segura, mas que serializa o acesso, criando um gargalo.
- **Vetor Compartilhado:** Uma estratégia "livre de locks" onde cada thread tem sua própria posição em um vetor. Evita o gargalo do `critical`, mas introduz um risco a nível de hardware.
- `rand()` **vs** `rand_r()`: `rand()` possui um estado global e usa um lock interno, serializando as chamadas. `rand_r()` é reentrante e permite o verdadeiro paralelismo na geração de números.

Análise de Desempenho: Resultados

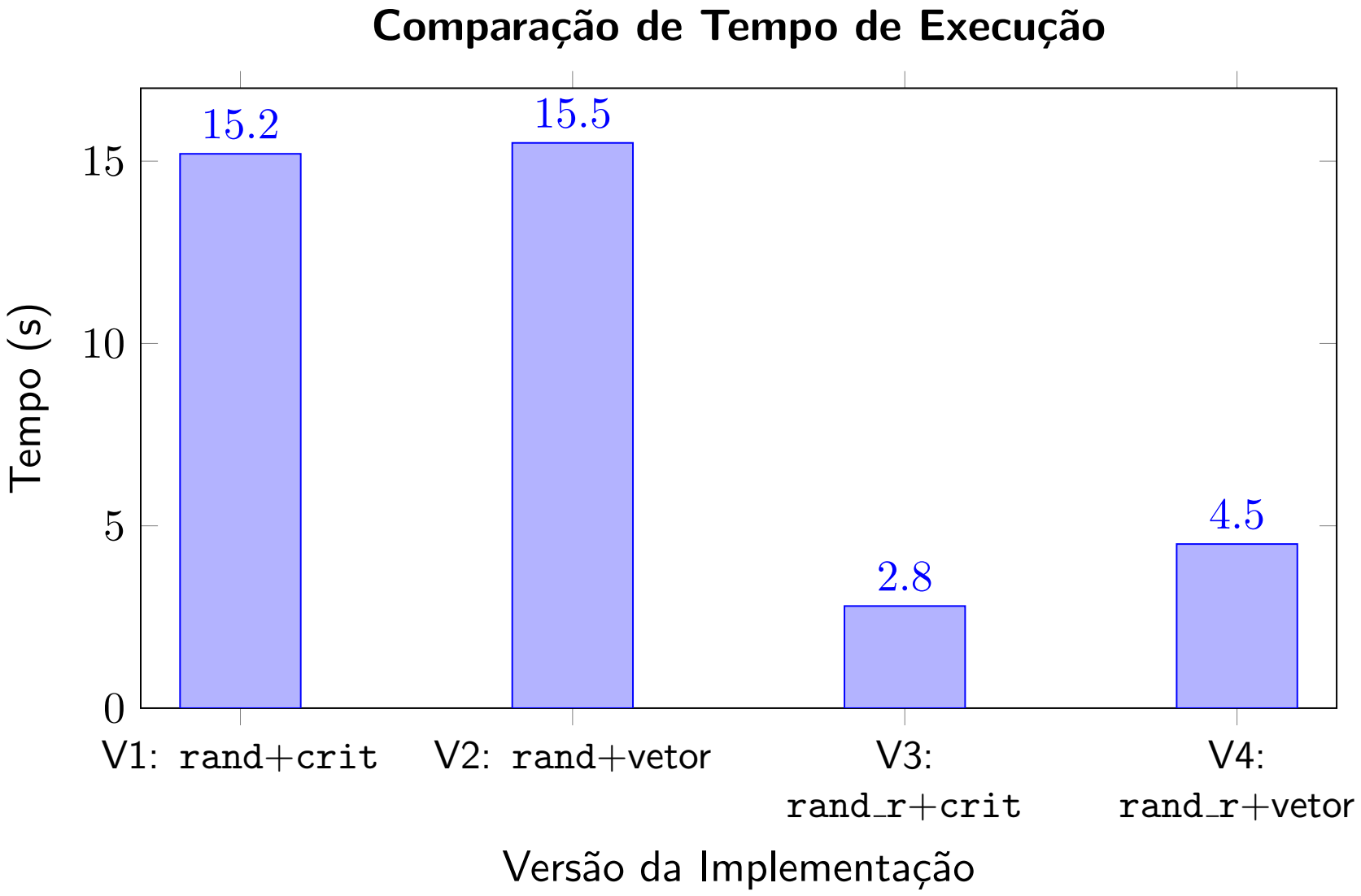


Figura: Resultados estimados para 100 milhões de passos.

Análise do Comportamento: O Falso Compartilhamento

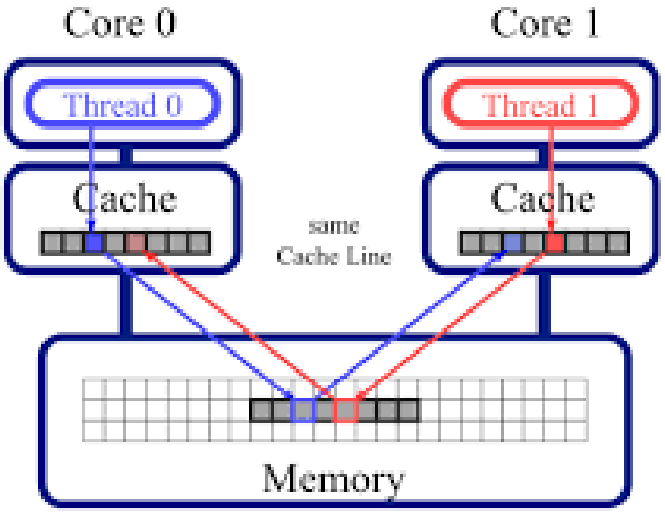
Pergunta: Por que a Versão 4 (vetor) é mais lenta que a Versão 3 (critical)?

A resposta é um fenômeno de hardware chamado **falso compartilhamento**.

- A CPU transfere dados da memória em blocos de 64 bytes (uma *cache line*).
- Um vetor de `long` (8 bytes) coloca os contadores de 8 threads na **mesma** linha de cache.
- Quando a Thread 0 modifica seu contador, a linha de cache inteira é invalidada para as Threads 1 a 7.
- Quando a Thread 1 vai modificar o seu, ela sofre um *cache miss* (operação lenta) e precisa recarregar a linha.
- Isso causa um "ping-pong" da linha de cache entre os núcleos, destruindo a performance.

Análise do Comportamento: O Falso Compartilhamento

- A CPU transfere dados da memória em blocos de 64 bytes (uma *cache line*).
- Um vetor de `long` (8 bytes) coloca os contadores de 8 threads na **mesma** linha de cache.
- Quando a Thread 0 modifica seu contador, a linha de cache inteira é invalidada para as outras threads.
- Isso causa um "ping-pong" da linha de cache entre os núcleos, destruindo a performance.



Coerência de Cache: Uma Perspectiva de Performance

A coerência de cache em sistemas multi-core, garante a consistência dos dados através de protocolos de invalidação ou atualização mantidos por hardware, usando o Snooping: O Mecanismo de "Escuta". Embora essencial para a correção do programa, o tráfego gerado por esses protocolos na interconexão do sistema — como mensagens de invalidação e transferências de linhas de cache — representa um gargalo de escalabilidade crítico e pode anular os ganhos de performance da paralelização.

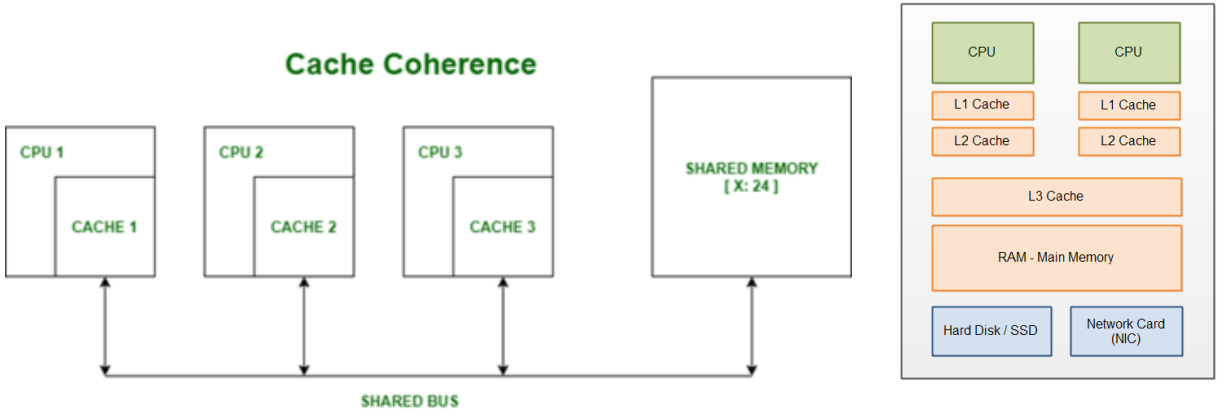


Figura: À esquerda: Coerência de cache. À direita: ilustração da arquitetura.

Conclusão

- O experimento demonstrou que a escolha de funções *thread-safe* (`rand_r`) é **essencial** para a escalabilidade do código paralelo.
- A eliminação de bloqueios de software (`critical`) não garante melhor desempenho. É preciso considerar a arquitetura de hardware.
- O **falso compartilhamento** pode ser um gargalo de desempenho mais severo do que uma seção crítica bem gerenciada.
- A versão mais performática (V3) foi aquela que combinou um algoritmo paralelo eficiente (`rand_r`) com uma sincronização de software simples e direta.

paralelo_rand.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 01//////////////////////////
////////////////////////////////////////////////////


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

//versão Paralela
void pi_paralel_for_critical_private() {

    long pontos_no_circulo_local = 0;

    #pragma omp parallel firstprivate(pontos_no_circulo_local)
    {
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand() / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                pontos_no_circulo_local++;
            }
        }
        #pragma omp critical
        {
            pontos_no_circulo_total += pontos_no_circulo_local;
        }

    } // Fim da região paralela
}

int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);
    start_time = omp_get_wtime();
    pi_paralel_for_critical_private();
    end_time = omp_get_wtime();
    double tempo_paralelo = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("Estimativa paralela de pi = %f\n", pi_estimado);
```

```
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo);

    return 0;
}

```

paralelo_rand_r.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 03//////////////////////////
////////////////////////////////////////////////////

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

//versão Paralela
void pi_paralel_for_critical_private() {
    unsigned int seed = time(NULL);
    long pontos_no_circulo_local = 0;

    #pragma omp parallel firstprivate(pontos_no_circulo_local, seed)
    {
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                pontos_no_circulo_local++;
            }
        }
        #pragma omp critical
        {
            pontos_no_circulo_total += pontos_no_circulo_local;
        }

    } // Fim da região paralela
}

int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);
    start_time = omp_get_wtime();
    pi_paralel_for_critical_private();
    end_time = omp_get_wtime();
    double tempo_paralelo = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("Estimativa paralela de pi = %f\n", pi_estimado);
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo);
```

```

    return 0;
}

```

paralelo_rand_vetor.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 02//////////////////////////
////////////////////////////////////////////////////

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

void pi_paralelo_vetor_compartilhado() {
    long *acertos_por_thread; // Ponteiro para o nosso vetor compartilhado
    int num_threads;

    // Pega o número de threads que o OpenMP decidiu usar
    num_threads = omp_get_max_threads();
    // Aloca o vetor com o tamanho exato do número de threads.
    // calloc já inicializa o vetor com zeros.
    acertos_por_thread = (long *)calloc(num_threads, sizeof(long));
    if (acertos_por_thread == NULL) {
        printf("Falha ao alocar memoria.\n");
        exit(1);
    }

    // Início da região paralela
    #pragma omp parallel
    {

        // Pega o identificador (ID) da thread atual
        int tid = omp_get_thread_num();

        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            // O gerador rand() não é "thread-safe".
            double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand() / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                // Cada thread incrementa APENAS a sua própria posição no vetor.
                acertos_por_thread[tid]++;
            }
        }
    } // Fim da região paralela

    for (int i = 0; i < num_threads; i++) {
        pontos_no_circulo_total += acertos_por_thread[i];
```

```c
    }

    // Libera a memória alocada para o vetor
    free(acertos_por_thread);
}


int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);

    // --- Executando a nova versão ---
    pontos_no_circulo_total = 0; // Reseta a contagem global
    start_time = omp_get_wtime();
    pi_paralelo_vetor_compartilhado();
    end_time = omp_get_wtime();

    double tempo_paralelo_vetor = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("\n--- Versao com Vetor Compartilhado ---\n");
    printf("Estimativa paralela de pi = %f\n", pi_estimado);
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo_vetor);

    return 0;
}

```

paralelo_rand_r_vetor.c

```c

////////////////////////////////////////////////////
///////////////VERSÃO 04//////////////////////////
////////////////////////////////////////////////////


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

// Definição global do número de passos para consistência
const long NUM_PASSOS = 100000000;
long pontos_no_circulo_total = 0;

void pi_paralelo_vetor_compartilhado() {
    long *acertos_por_thread; // Ponteiro para o nosso vetor compartilhado
    int num_threads;

    // Pega o número de threads que o OpenMP decidiu usar
    num_threads = omp_get_max_threads();
    // Aloca o vetor com o tamanho exato do número de threads.
    // calloc já inicializa o vetor com zeros.
    acertos_por_thread = (long *)calloc(num_threads, sizeof(long));
    if (acertos_por_thread == NULL) {
        printf("Falha ao alocar memoria.\n");
        exit(1);
    }

    // Início da região paralela
    #pragma omp parallel
    {
        num_threads = omp_get_max_threads();
        // Pega o identificador (ID) da thread atual
        int tid = omp_get_thread_num();
        unsigned int seed = time(NULL) + tid;
        // A diretiva 'for' distribui as iterações entre as threads.
        // Cada thread executa seu próprio subconjunto de 'i'.
        #pragma omp for
        for (long i = 0; i < NUM_PASSOS; i++){
            // O gerador rand() não é "thread-safe".
            double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
            double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;

            if (x * x + y * y < 1.0) {
                // Cada thread incrementa APENAS a sua própria posição no vetor.
                acertos_por_thread[tid]++;
            }
        }
    } // Fim da região paralela
```

```c

    // Soma os resultados parciais de cada thread para obter o total.
    for (int i = 0; i < num_threads; i++) {
        pontos_no_circulo_total += acertos_por_thread[i];
    }

    // Libera a memória alocada para o vetor
    free(acertos_por_thread);
}


int main() {
    double start_time, end_time;

    printf("Iniciando analise de desempenho para %ld passos.\n", NUM_PASSOS);

    // --- Executando a nova versão ---
    pontos_no_circulo_total = 0; // Reseta a contagem global
    start_time = omp_get_wtime();
    pi_paralelo_vetor_compartilhado();
    end_time = omp_get_wtime();

    double tempo_paralelo_vetor = end_time - start_time;
    double pi_estimado = 4.0 * pontos_no_circulo_total / NUM_PASSOS;

    printf("\n--- Versao com Vetor Compartilhado ---\n");
    printf("Estimativa paralela de pi = %f\n", pi_estimado);
    printf("Tempo Paralelo: %f segundos\n", tempo_paralelo_vetor);

    return 0;
}

```